



Java程序设计

Ch02-1 Java语言基础

余 烨

本章学习提示

- 本章的重点是介绍Java的语言基础，主要包括：
  - Java中的基本数据类型
  - 数据类型的优先关系和相互转换规则
  - 字符编码及标识符
  - 运算符和表达式
  - 程序流程控制
  - 数组

目　录

1 基本数据类型

2 数据类型转换

3 字符集与标识符

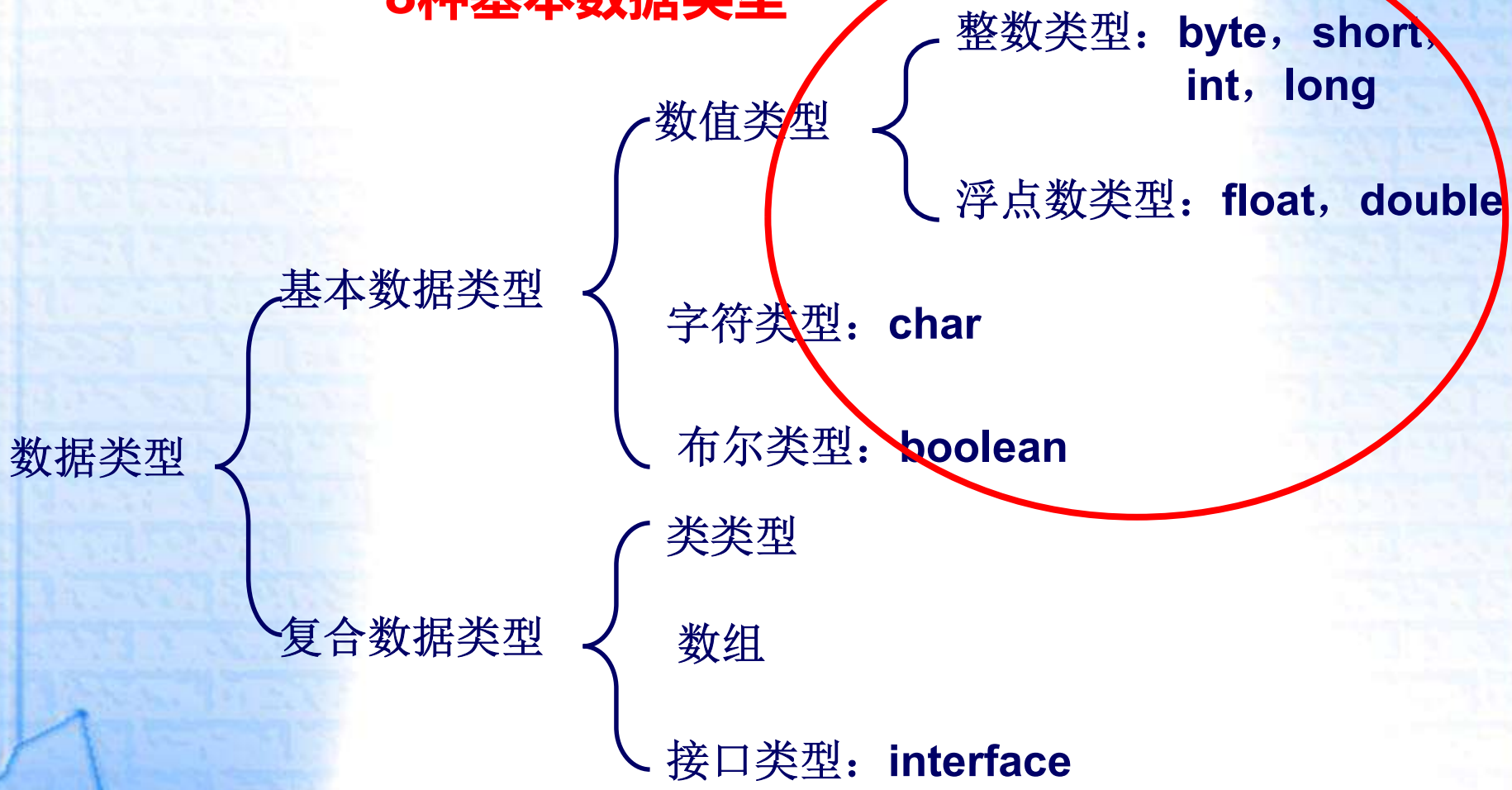
4 运算符和表达式

5 程序流程控制

6 数　组

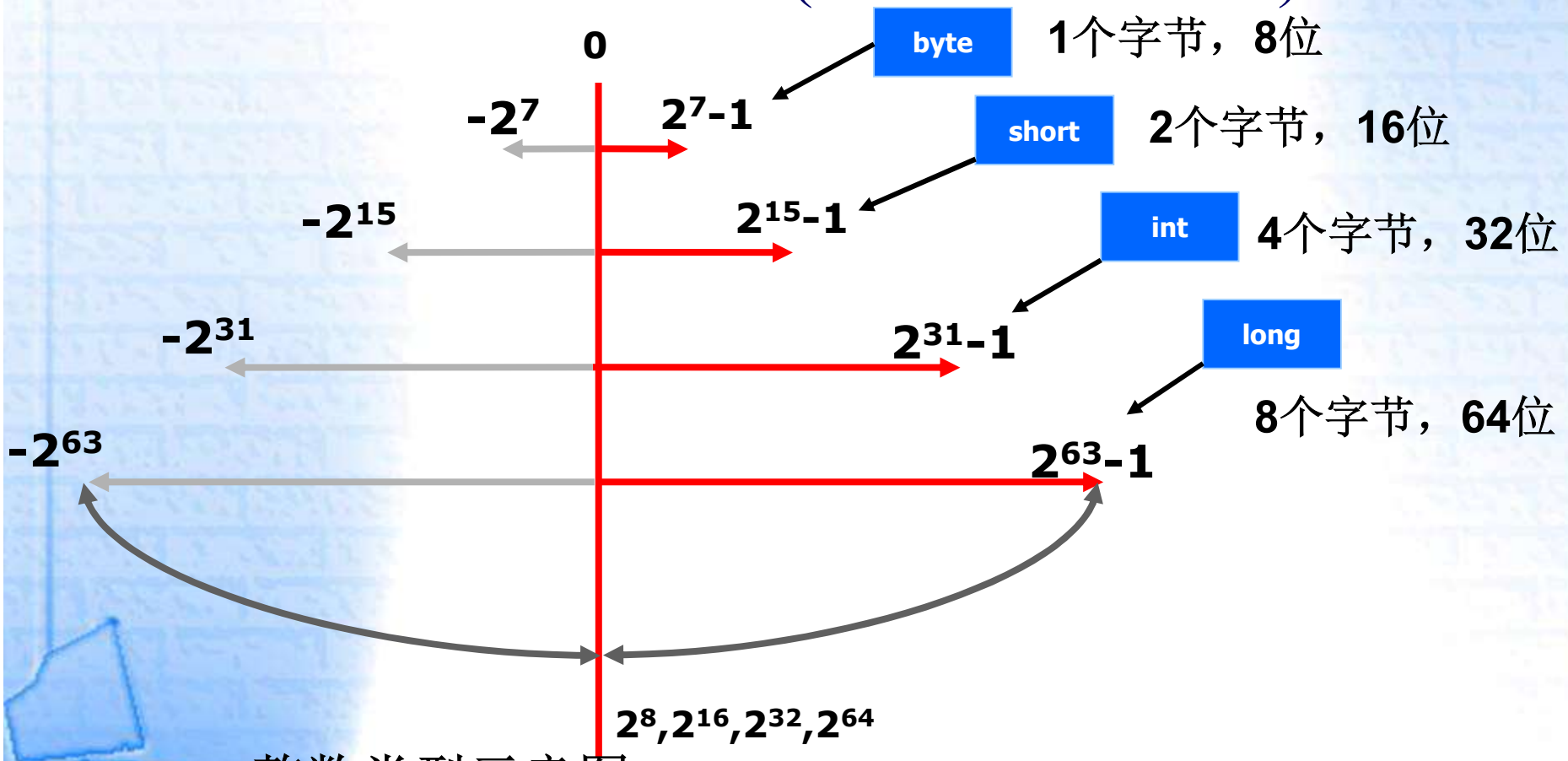
Java语言的数据类型

8种基本数据类型



整数类型

- Java编程语言中的整数类型都是有符号整数，不存在无符号整数。(与C和C++不同)



整数类型示意图（各种整型的特点和使用范围）

整数类型

- 整型可用十进制、八进制或十六进制表示。首位为“0”表示八进制的数值；首位为“0x”表示十六进制的数值。

  请看下面的例子：

  - 5　　表示十进制值5
  - 075　　表示八进制数值75（也就是十进制数61）
  - 0x9ABC　　表示十六进制的数值9ABC（也就是十进制数39612）

- 整型，如在其后有一个字母“L”表示一个long常量（也可以用小写“l”）

补

原码

将最高位作为符号位（以0代表正，1代表负），其余各位代表数值本身的绝对值（以二进制表示）。

为了简单起见，我们用1个字节来表示一个整数。

+7的原码为：　　00000111

-7的原码为：　　10000111

问题：

+0的原码为：　　00000000

-0的原码为：　　10000000

反码

一个数如果为正，则它的反码与原码相同；一个数如果为负，则符号位为1，其余各位是对原码取反。

为了简单起见，我们用1个字节来表示一个整数。

+7的反码为：　　00000111

-7的反码为：　　11111000

问题：

+0的反码为：　　00000000

-0的反码为：　　11111111

补充知识——原码、补码&反码

一个数如果为正，则它的原码、反码、补码相同；一个数如果为负，则符号位为1，其余各位是对原码取反，然后整个数加1。

为了简单起见，我们用1个字节来表示一个整数。

+7的补码为：00000111

-7的补码为：第①步11111000

第②步 +1

11111001

0的补码表示：

+0的补码：00000000

-0的补码：第①步 11111111

第②步 +1

1 00000000

第③步 进位1被丢弃

补充知识——原码、补码&反码

已知一个负数的补码，将其转换为十进制数，步骤：

1、先对各位取反；

2、将其转换为十进制数；

3、加上负号，再减去1。

例如：

11111010，最高位为1，是负数，先对各位取反得00000101，转换为十进制数得5，加上负号得-5，再减1得-6。

- 数据的内存存储状态如下
  - 最高位是符号位，正数使用原码表示，最高位为0；负数用补码表示，最高位为1

问题

- 为什么byte的表示范围是-128~127，而不是-128~128？
  - ✓ 1个字节8位，可以表示2的8次方，即256个不同的数，而-128~127刚好有256个数；
  - ✓ 正数能表示的范围：00000000~01111111，即0~127；
  - ✓ 负数能表示的范围：10000000到11111111，是补码，转化为十进制，即-128～-1

浮点类型

- Java语言中有两种浮点类型：float和double.
  - 如果一个数值常量中包含小数点或指数部分，或者其后跟有字母F或f（float），D或d（double），则该数为浮点数.
  - 如果不明确指明浮点数的类型，浮点数缺省为double类型.

    3.14159　(double型浮点数)　→ 十进制表示法

    2.08E25　(double型浮点数)　→ 科学计数法

    6.56f　(float型浮点数)
  - float为32位（单精度），double为64位（双精度）

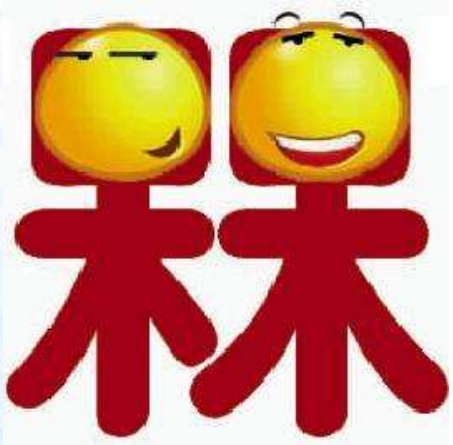
字符型

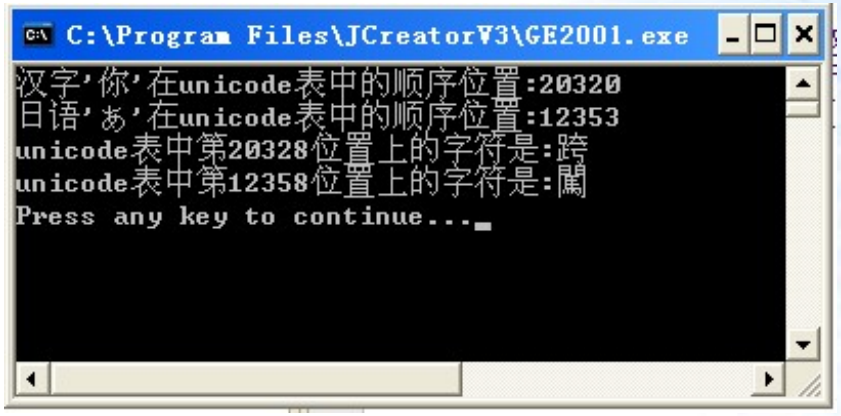
- char
  - char是2个字节（16位）的无符号型数据
  - 字符必须用一对单引号括起来，如‘a’，‘B’等
  - Unicode字符集采用双字节对字符进行编码
  - 与C语言类似，Java也提供转义字符，以反斜杠（\）开头，将其后的字符转变为另外的含义。

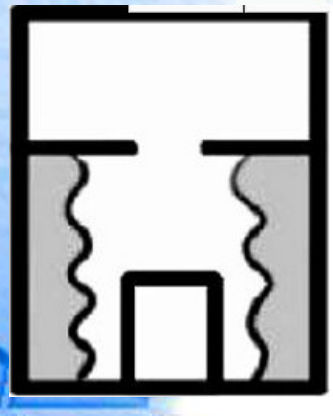
\n	换行	\t	水平制表
\b	退格	\r	回车
\\	反斜线	\’	单撇号

字符型之例

- 请查找‘a’、‘粽’、‘囧’ 在unicode中的顺序位置，并查明unicode中位于29233、24680位置的字符是什么。









example02_01.java

字符串

- 值得注意的一点是，用双引号引用的文字，就是我们平时所说的字符串类型（String），它不是基本类型，而是一个对象类型，Java是将字符串作为对象实现的.

```
char ch = 'x';
String str = "x";
```

- 为char类型所规定的字符转义序列在字符串内同样适用，如“three\nlines”等.
- 字符串之间的连接：（“Str 1” + “Str 2”）.

逻辑类型

- 逻辑类型
  - 取值： true ， false
  - 变量的定义

    使用关键字boolean来定义逻辑变量，

    定义时也可以赋给初值：

    boolean x=true，tom=false，jiafei;
  - boolean x = 1;　boolean y = 0;



```
if (1)
   {   System.out.println("true");   }
else if (0)
   {   System.out.println("false");  }
```



实践经验：

boolean is, done;

语句：if ((is == true) & (done == false))

建议写成：if ((is) & (!done))

复合数据类型

- 复合数据类型：
  - 类、数组、接口类型等
  - 由一系列基本数据类型组成的
  - 用户根据需要可以自定义复合类型
  - Java中没有C、C++中的结构类型、联合类型、枚举类型和指针类型。

基本数据 vs. 数据类型类

- 对于每一种基本数据类型，Java分别提供相应的类对其进行封装，以便提供更强大的数据处理能力

```
int a = Integer.parseInt("3");
Float f = Float.MAX_VALUE;
```

基本 数据类型	数据 类型类
byte	Byte
short	Short
int	Integer
long	Long
char	Char
boolean	Boolean
float	Float
double	Double

数据类型的默认值

- 为了提高程序的健壮性，JAVA为所有的数据类型（成员变量）设置了默认值

boolean	false
char	‘\000’（NUL，空）
byte	(byte)0
short	(short)0
int	0
long	0L
float	0.0f
double	0.0

目　录

1 基本数据类型

2 数据类型转换

3 字符集与标识符

4 运算符和表达式

5 程序流程控制

6 数　组

类型转换

- Java是一种强类型的语言(Strong typed language)
  - 任何变量必须定义为某种数据类型，每个变量的类型说明是唯一的
  - 赋值运算之前，要检查运算符左右两端类型是否一致
  - 方法调用的时候，要求实参与形参的类型一致
- 数据类型转换有2种：
  - 低级到高级的自动类型转换
  - 高级到低级的强制类型转换

自动类型转换

- 容量小的类型（低级）自动转换为容量大（高级）的数据类型；数据类型按容量大小排列为（从低级到高级）：
  - **byte, short, char--int--long--float—double**
  - **byte, short, char**之间不会相互转换，他们三者在计算时首先转化为int类型.

    如：int x=7; ‘B’+x 的结果是什么类型？

    （**char型数据**和**整型数据**运算结果的精度是**int** ）
- Java将按运算符两边的操作元的最高精度保留结果的精度
  - 例如：

    5/2的结果是2，要想得到2.5，必须写成5.0/2，或5/2.0

强制数据类型转换

- 由低级向高级可以自动转换，但是由高级向低级转换必须显示说明、进行强制转换.

```
short s = 1 ;
s = s+1 ;
System.out.println(s);
```

✗ → `s=(short) (s+1);`

↓

`s+=1;` →

In Java规范：

e1+=e2; ⟺ e1=(T1)(e1+e2);

Where T1 is the Type of e1

```
byte b;
b=3;
b = b*3;
System.out.println(b);
```

```
--------------------Configuration: <Default>-------------
D:\JavaTest\HelloWorld.java:7: 错误: 不兼容的类型:
 从int转换到byte可能会有损失
                b = b*3;   //(byte)(b*3)
                  ^
```

```
byte c = 255;
int a = c;
System.out.println(a);
```

```
--------------------Configuration: <Default>--------------------
D:\JavaTest\Hello.java:9: 错误: 不兼容的类型: 从int转换到byte可能会有损失
        byte c = 255;
                 ^
1 个错误

Process completed.
```

```
byte c = (int)255;
int a = c;
System.out.println(a);
```

高级变量转换为低级变量之例

```
//简单类型强制转换
double a = 346.73;
double b = 345.1;
int c = (int) a;
int d = (int) b;
System.out.println(c);
System.out.println(d);
```

```
//int与字节类型的转换
int c = 355;
int d = 483;
byte c1 = (byte)c;
byte d1 = (byte)d;
System.out.println(c1);
System.out.println(d1);
```

Integer.toBinaryString(int);

example02_02.java

目　录

1 基本数据类型

2 数据类型转换

3 字符集与标识符

4 运算符和表达式

5 程序流程控制

6 数　组

字符集 – 编码方式

- **ASCII码**
  - 上个世纪60年代，美国制定了一套字符编码，对英语字符与二进制位之间的关系，做了统一规定。这被称为ASCII码，一直沿用至今。
  - ASCII码一共规定了128个字符的编码，比如空格"SPACE"是32（二进制00100000），大写的字母A是65（二进制01000001）。这128个符号（包括32个不能打印出来的控制符号），只占用了一个字节的后面7位，最前面的1位统一规定为0。
- **Unicode码**
  - 可以想象，如果有一种编码，将世界上所有的符号都纳入其中。每一个符号都给予一个独一无二的编码，那么乱码问题就会消失。这就是Unicode，就像它的名字都表示的，这是一种所有符号的编码。

Java采用的编码——Unicode

- “Universal Multiple-Octet Coded Character Set”，简称为UCS。
- 如果把各种文字编码形容为各地的方言，那么Unicode就是世界各国合作开发的一种语言，将世界上所有的文字用2个字节统一进行编码。

Unicode只有一个字符集，中、日、韩的三种文字占用了Unicode中0x3000到0x9FFF的部分 Unicode目前普遍采用的是UCS-2,它用两个字节来编码一个字符， 比如汉字"经"的编码是0x7ECF,注意字符编码一般用十六进制来 表示，为了与十进制区分，十六进制以0x开头，0x7ECF转换成十进制 就是32463,UCS-2用两个字节来编码字符，两个字节就是16位二进制， 2的16次方等于65536,所以UCS-2最多能编码65536个字符。 编码从0到127的字符与ASCII编码的字符一样，比如字母"a"的Unicode 编码是0x0061,十进制是97,而"a"的ASCII编码是0x61,十进制也是97, 对于汉字的编码，事实上Unicode对汉字支持不怎么好，这也是没办法的， 简体和繁体总共有六七万个汉字，而UCS-2最多能表示65536个，才六万 多个，所以Unicode只能排除一些几乎不用的汉字，好在常用的简体汉字 也不过七千多个，为了能表示所有汉字，Unicode也有UCS-4规范，就是用 4个字节来编码字符

UTF-8

- Unicode只是一个符号集，它只规定了符号的二进制代码，却没有规定这个二进制代码应该如何存储。
- 互联网的普及，强烈要求出现一种统一的编码方式。UTF-8就是在互联网上使用最广的一种Unicode的实现方式。其他实现方式还包括UTF-16（字符用两个字节）和UTF-32（字符用四个字节表示），不过在互联网上基本不用。
- UTF-8最大的一个特点，就是它是一种变长的编码方式。它可以使用1~4个字节表示一个符号，根据不同的符号而变化字节长度。

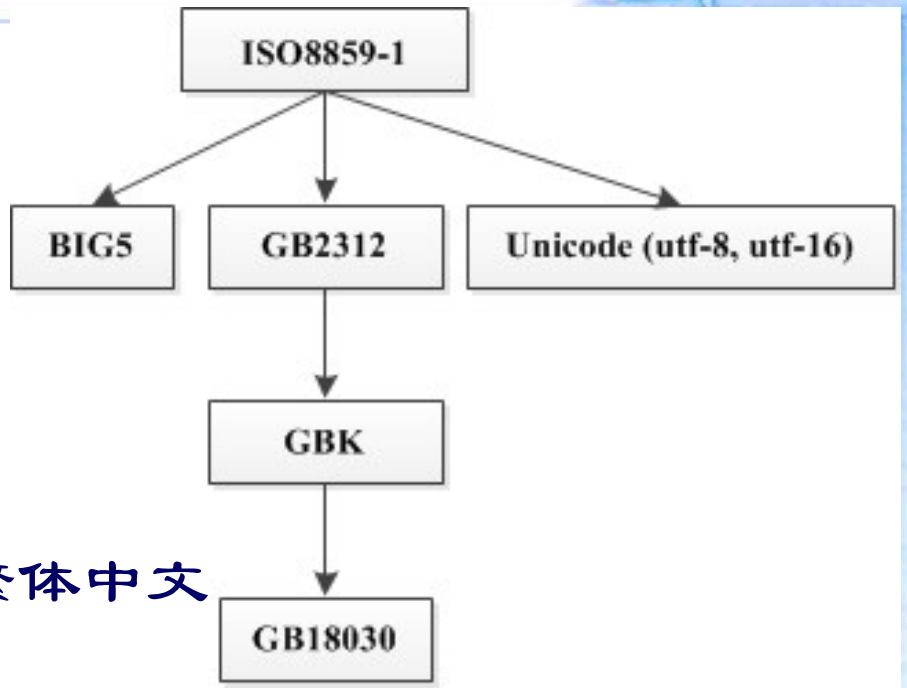
UTF-8

- UTF-8的编码规则很简单，只有二条：
  - 对于单字节的符号，字节的第一位设为0，后面7位为这个符号的unicode码。因此对于英语字母，UTF-8编码和ASCII码是相同的。
  - 对于n字节的符号（n>1），第一个字节的前n位都设为1，第n+1位设为0，后面字节的前两位一律设为10。剩下的没有提及的二进制位，全部为这个符号的unicode码。

Unicode/ UCS-4	bit数	**UTF-8**	byte数
0000 ~ 007F	0~7	**0**XXX XXXX	1
0080 ~ 07FF	8~11	**110**X XXXX **10**XX XXXX	2
0800 ~ FFFF	12~16	**1110**XXXX **10**XX XXXX **10**XX XXXX	3
1 0000 ~ 1F FFFF	17~21	**1111 0**XXX **10**XX XXXX **10**XX XXXX **10**XX XXXX	4

其他编码方式

- ISO8859-1
  - 西欧字符集
- GB2312
  - 大陆使用最早、最广的简体中文字符集
- GBK
  - GB2312的扩展，可以表示繁体中文
- GB18030
  - 最新GBK的扩展，中国所有非手持/嵌入式计算机系统的强制实施标准，可以表示汉字、维吾尔文、藏文等
- BIG5
  - 台湾、香港等地区使用，表示繁体字



一些网页的编码方式




```
<meta http-equiv=Content-Type
content="text/html;charset=utf-8">
```

一些网页的编码方式


网易 证
http://www.163.com/
尚学堂_高淇java零
Deep Lea
应用
设为首页
網易 NETEASE
www.163.com
后退(B)
前进(O)
刷新(R)
发送到手机
收藏到云盘
网页另存为(S)...
添加到收藏夹(F)...
全选(A)
打印(I)...
编码(E)
查看源代码(V)
使用迅雷下载全部链接
审查元素(N)
自动检测
Unicode (UTF-8)
中文（简体）(GBK)
中文（简体）(gb18030)
Unicode (UTF-16LE)
阿拉伯文 (Windows-1256)
阿拉伯文 (ISO-8859-6)
北欧语言 (ISO-8859-10)
波罗的海周边语言 (ISO-8859-4)
波罗的海周边语言 (ISO-8859-13)
波罗的海周边语言 (Windows-1257)
韩语
凯尔特语 (ISO-8859-14)
罗马尼亚文 (ISO-8859-16)
南欧语言 (ISO-8859-3)
日文 (Shift_JIS)
日文 (EUC-JP)
日文 (ISO-2022-JP)
泰文
土耳其文
希伯来文 (Windows-1255)
希伯来文 (ISO-8859-8-I)
希伯来文 (ISO-8859-8)


https://www.taobao.com/
Deep Learning Su
百度一下，你就
手机逛淘宝
苹果6低至3688元
用天猫魔盒看
日特卖
后退(B)
前进(O)
刷新(R)
发送到手机
收藏到云盘
网页另存为(S)...
添加到收藏夹(F)...
全选(A)
打印(I)...
编码(E)
查看源代码(V)
使用迅雷下载全部链接
自动检测
Unicode (UTF-8)
中文（简体）(GBK)
中文（简体）(gb18030)
Unicode (UTF-16LE)
阿拉伯文 (Windows-1256)
阿拉伯文 (ISO-8859-6)
北欧语言 (ISO-8859-10)
韩语
凯尔特语 (ISO-8859-14)
罗马尼亚文 (ISO-8859-16)
南欧语言 (ISO-8859-3)
日文 (Shift_JIS)
日文 (EUC-JP)
日文 (ISO-2022-JP)
泰文

标识符

- 标识符identifier 好像一个人生下来就要给他起个名字 一样，作为一种识别记号。计算机中的变量、常量、方法、类等也都是用名字来加以识别的，这种名字就是标识符。
- Java关于标识符的语法规则：
  1. 由字母、下划线_、美元$符号和数字组成，名字长度不限
  2. 第一个字符不能是数字符号
  3. 注意英文字母大小写代表不同含义
  4. 取名应遵循易于理解、便于记忆的原则

例如：合法标识符 **Lotus_1　wang$**

非法标识符 **A?2　3ab**

在C、C++里，标识符由字母、数字和下划线三种字符组成，且第一个字符必须为字母或下划线。（区分大小写）

Java中标识符命名规则

- 类名和接口名
  - 首字母大写，含有大小写，如：MyClass, HelloWorld等
- 方法名、变量名
  - 首字母小写，分隔用字首字母大写，尽量不用下划线，少用$符号，如：setDistance，myName
- 常量名
  - 基本数据类型的常量名全部大写，字与字之间用下划线分隔。对象常量可大小混写，如：SIZE_NAME

保留字

- 保留字（关键字）就是Java语言中已经被赋予特定意义的一些单词
- 不可以把这类词作为名字来用
- 注意
  - goto和const是Java编程语言中保留的没有意义的关键字
  - 所有的java关键字都是小写英文
  - Java所有数据类型的长度和表示是固定的，与平台无关，不是像在C/C++语言中那样数据类型的长度根据不同的平台而变化。这正是Java语言的一大特点.

保留字

abstract	boolean	break	byte	case
catch	char	class	const	continue
do	double	else	extends	false
finally	cast	default	final	finally
float	for	future	generic	goto
if	implements	import	inner	instanceof
int	interface	long	native	new
null	operator	outer	package	private
protected	public	rest	return	short
static	super	switch	synchronized	this
throw	throws	transient	true	try
var	void	volatile	while	

目　录

1 基本数据类型

2 数据类型转换

3 字符集与标识符

4 运算符和表达式

5 程序流程控制

6 程序流程控制

Java的运算符

- 按运算符对数据的运算结果分类：
  - 算术运算符、逻辑运算符、关系运算符、赋值运算符和位运算符
- 按运算符运算的数据个数分类：
  - 一元运算符、二元运算符和三元运算符.
- 算术运算符有五种 加 + 减 – 乘 * 除 / 取余%
  其中除减号可作为一元运算符外均为二元运算符.
  - 另外两个经常使用的一元运算符是
    - i++ 和 ++i
    - i-- 和 --i

程序分析（1）

```
int a=1, b=2;
 a+++b = ?  3
 a = ?  2
int i=3;
 System.out.println(i++);  3
 System.out.println(i);  4
int i=3;
 int count1 = ( (i++) + (i++) + (i++) );
 System.out.println(i);  6
 System.out.println(count1);  12
 i=3;
 int count2 = ( (++i) + (++i) + (++i) );
 System.out.println(i);  6
 System.out.println(count2);  15
```

```
//算术运算测试
public class example02_03 {
public static void main(String
args[]) {
    int a=4;
    int b=10;
    int d=b*++a;
    int e=(a/(++b));
    System.out.println("a="+a);
    System.out.println("b="+b);
    System.out.println("d="+d);
    System.out.println("e="+e);
          }   }
```

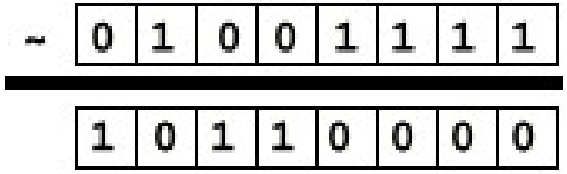
语句与表达式

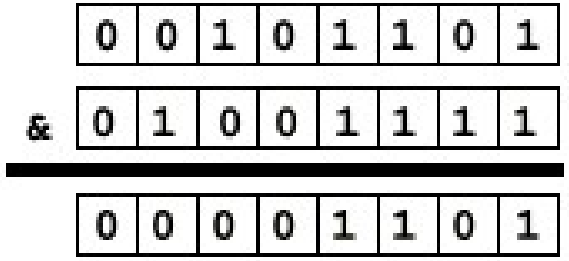
- 语句是构成程序的基本单位，语句具有独立完整的含义，可以对计算机发出操作命令，每一条语句都必须以分号；作为结束符.
- 表达式是用运算符把操作数（变量、常量和方法）连接起来表达某种运算或含义的式子.
- 一个表达式可以同时包括多个操作，而操作的顺序由各运算符的优先级及括号来决定。其中运算符是算术运算符称为算术表达式；运算符为逻辑运算符称为逻辑表达式.
- 语句和表达式的区别与联系
  - 语句是程序的组成部分；表达式是语句的组成部分
  - 语句有结束符；表达式没有结束符
  - 语句是针对程序而言的；表达式是数学上的计算概念
  - 算式的计算结果=号在右边；赋值语句=号在表达式左边

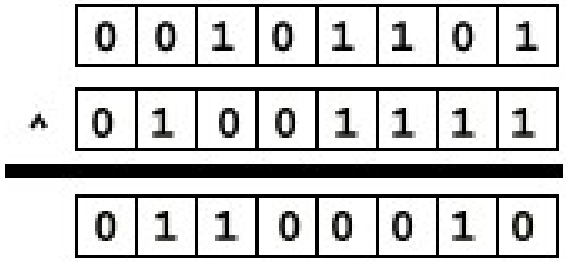
位运算符

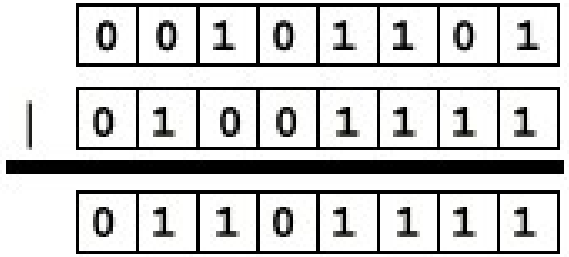
运算符	运算	举例	含义
～	位反	～x	将x逐位取反
&	位与	x&y	x、y 逐位进行与操作
|	位或	x|y	x、y 逐位进行或操作
^	位异或	x^y	x、y 逐位进行 相同取0 相异取1
<<	左移	x<<y	x向左移动，位数是y
>>	右移	x>>y	x向右移动，位数是y
>>>	不带符号右移	x>>>y	x向右移动，位数是y，空位补0

位运算符


~ 0 1 0 0 1 1 1 1
1 0 1 1 0 0 0 0


0 0 1 0 1 1 0 1
& 0 1 0 0 1 1 1 1
0 0 0 0 1 1 0 1


0 0 1 0 1 1 0 1
^ 0 1 0 0 1 1 1 1
0 1 1 0 0 0 1 0


0 0 1 0 1 1 0 1
| 0 1 0 0 1 1 1 1
0 1 1 0 1 1 1 1

移位运算符

数	X	X<<2	X>>2	X>>>2
17	00010001	? 68	? 4	?
-17	11101111	? -68	? -5	?

左移：低位补**0**；
右移：正数，高位补**0**；负数，高位补**1**；
无符号右移：高位均补**0**；

位运算应用

- “清零” -- 和“零”按位“与”
- “乘 2” -- 左移“<<”一位
- “除 2” -- 右移“>>”一位
- “交换变量” -- 通过按位异或“^”
- “加密”

1. a = 10100001, b = 00000110
2. a = a ^ b; //a = 10100111
3. b = a ^ b; //b = 10100001
4. a = a ^ b; //a = 00000110

位运算应用– 加密

```
public class example02_06 {
   public static void main(String [] args) {
      char a1='合',a2='肥',a3='工',a4='大';
      char secret='0';
      a1=(char)(a1^secret);        //21512 ^ 54
                  //(101010000001000 ^ 110110)
      a2=(char)(a2^secret);        //32933
      a3=(char)(a3^secret);        //24037
      a4=(char)(a4^secret);        //22823
      System.out.println("密文:"+a1+a2+a3+a4);
      a1=(char)(a1^secret);
      a2=(char)(a2^secret);
      a3=(char)(a3^secret);
      a4=(char)(a4^secret);
      System.out.println("原文:"+a1+a2+a3+a4);
   } }
```

example02_06.java



二元算术运算符、逻辑运算符和位运算符

运算符	运算	举例	含义	运算符	运算	举例	含义
+=	加法	x +=y	x=x+y	&=	与	x &=y	x=x&y
-=	减法	x-=y	x=x-y	|=	或	x |=y	x=x|y
*=	乘法	x*=y	x=x*y	^=	异或	x ^=y	x=x^y
/=	除法	x/=y	x=x/y	<<=	左移	x <<=y	x=x<<y
%=	取余	x%=y	x=x%y	>>=	右移	x>>=y	x=x>>y
>>>=	不带符号右移					x >>>=y	x=x>>>y

运算符的优先级

运算符	描述	优先级	结合性
. [ ] ( )	域、数组、括号	1	从左至右
++ -- - ! ~	一元操作符	2	从右至左
* / %	乘、除、取余	3	从左至右
+ -	加、减	4	从左至右
<< >> >>>	位运算	5	从左至右
< <= > >=	逻辑运算	6	从左至右
== !=	逻辑运算	7	从左至右
&	按位与	8	从左至右
^	按位异或	9	从左至右
|	按位或	10	从左至右
&&	逻辑与	11	从左至右
||	逻辑或	12	从左至右
?:	条件运算符	13	从右至左
= *= /= %= += -= ， <<= >>= >>>= &= ^= |=	赋值运算符	14	从右至左

目　录

1 基本数据类型

2 数据类型转换

3 字符集与标识符

4 运算符和表达式

5 程序流程控制

6 数　组

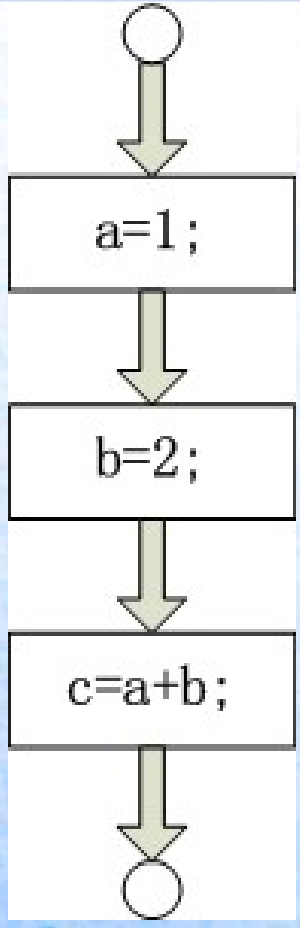
Java语言控制结构

- 顺序结构
- 分支结构
- 循环结构
- break和continue语句的用法

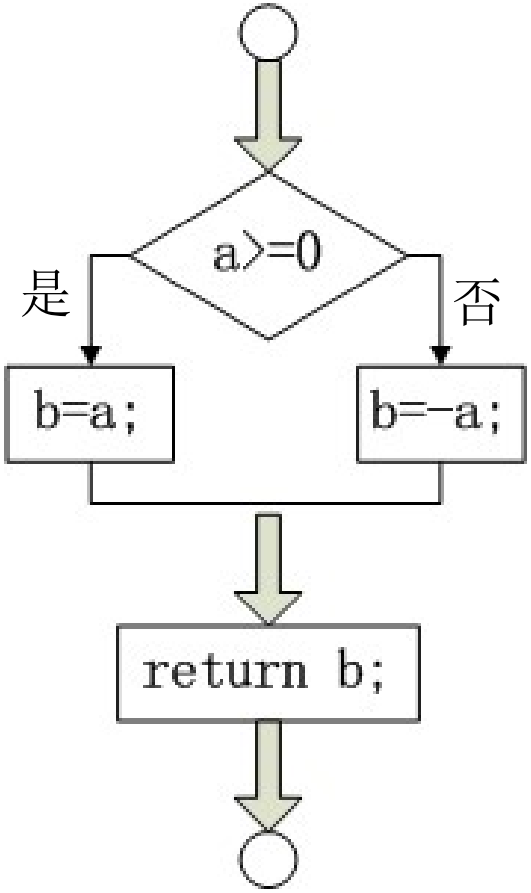
基本控制结构

- 结构化程序设计的原则及三种基本结构
  - 原则：1. 自顶向下
    2. 逐步求精
    3. 模块化
    4. 限制使用goto语句
  - 三大流程控制结构
    - 顺序结构
    - 分支结构（选择结构）
    - 循环结构

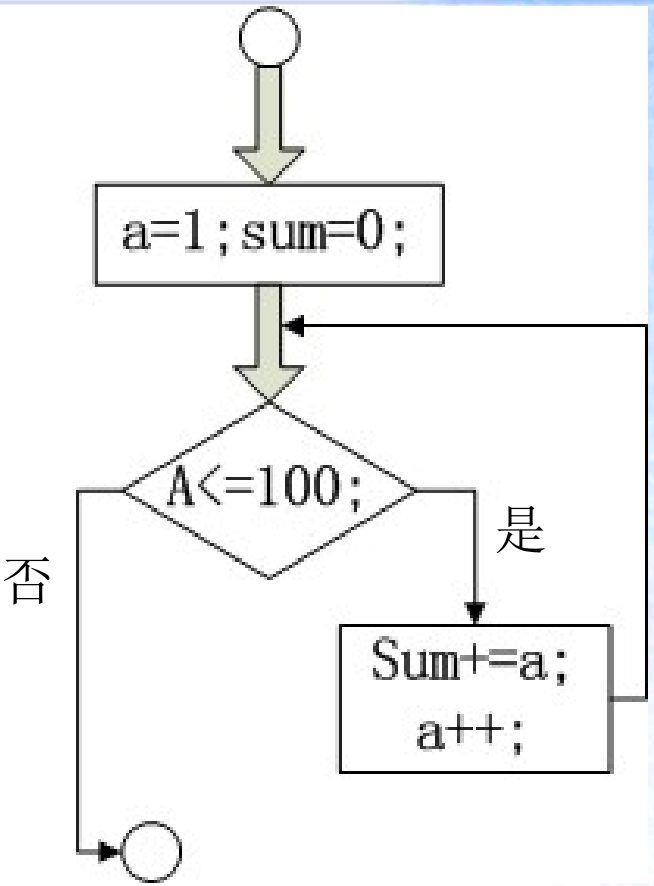
基本控制结构



顺序结构
（Sequence Structure）



选择结构
(Selection Structure)



循环结构
(Repetition Structure)

顺序结构

- 让程序可以顺序的执行

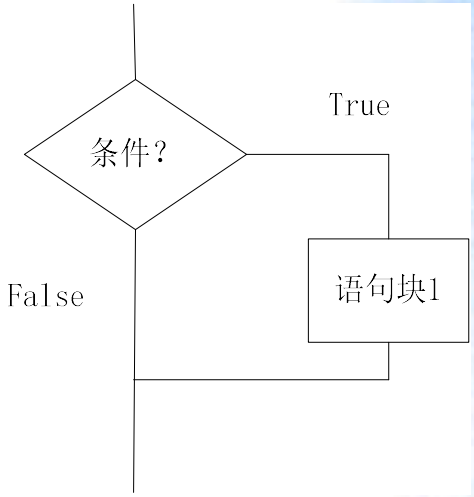


```
int a = 7;
System.out.println(“a=”+a);
System.out.println(“Hello!”);
a++;
System.out.println(“a=” +a );
a--;
System.out.println(“a=” +a );
```

分支结构

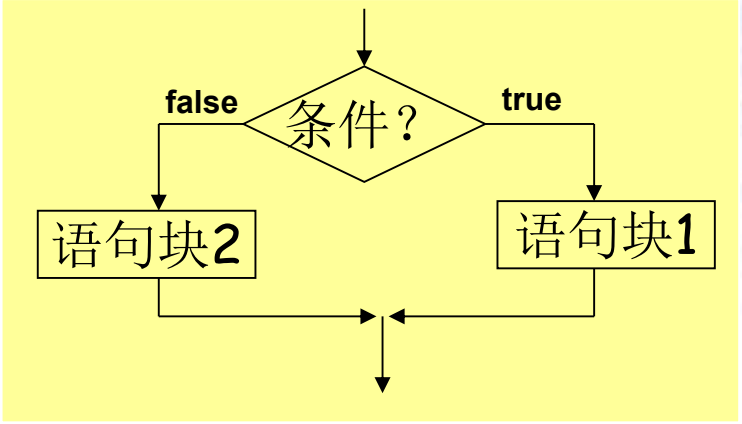
- 分支结构，是在两条或多条（两条以上）执行路径中选择一条执行的控制结构
- 单分支

```
if（条件表达式）{
   语句组；  }
```



- 双分支

```
if(条件) {(语句块1)
1.   }
2.   else  {
3.       (语句块2)
4.   }
```



分支结构

- 多分支

```
if（条件表达式1）{ 语句组1; }
else if （条件表达式2） { 语句组2; }
……
else { 语句组3; }
```

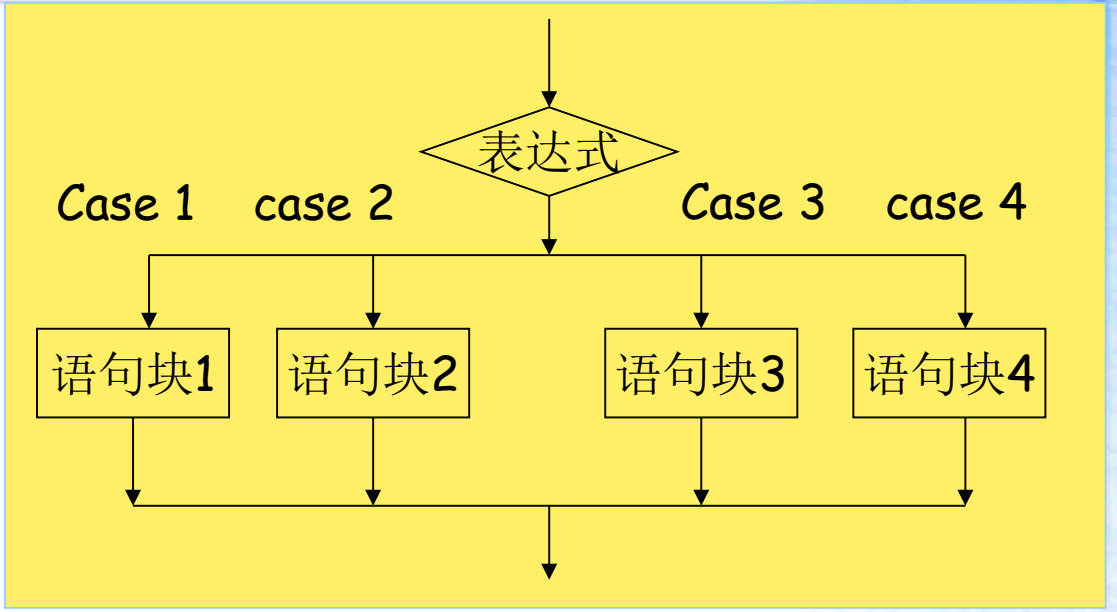
- 例子

编写一个程序，可以输入学生的平均分数，如果分数高于95，则输出“恭喜您获得了国家奖学金”；如果分数在80-95之间，则输出“恭喜您获得了等级奖学金”；否则，输出“抱歉，你未获得任何奖学金！”

Example3_01

Switch 语句

```
Switch(表达式)
 {
  case 值1 :  语句块1 ;
              break ;
  case 值2 :  语句块2 ;
              break ;
  case 值3 :  语句块3 ;
              break ;
  ……
  case 值n :  语句块n ;
              break ;
 default:
              语句块n+1;
 }
```



1. Switch 语句中的表达式的数据类型可以是byte、char、short、int类型，不允许浮点型和long型。
2. 根据表达式值与case语句后面的匹配情况决定程序执行的分支
3. 每个case 语句都要有break语句
4. 不匹配的情况执行 default 语句

Switch语句

- 例子
  - 请输入您需要购买的玫瑰花支数，我会告诉您它的含义。
    - 1：你是我的唯一
    - 3：我爱你
    - 4：天长地久
    - 11：爱你，一心一意
    - else：means nothing

Example3_04

下面代码会产生什么样的输出？

```
public class test03_01 {
    public static void main(String [] args){
        char letter = 'A' ;
        switch (letter){
            case 'A':
            case 'a':
                System.out.println("Some kind of A.");
            case 'B':
            case 'b':
                System.out.println("Some kind of B.");
                break;
            default:
                System.out.println("Something else.");
                break;
        }
    }
}
```

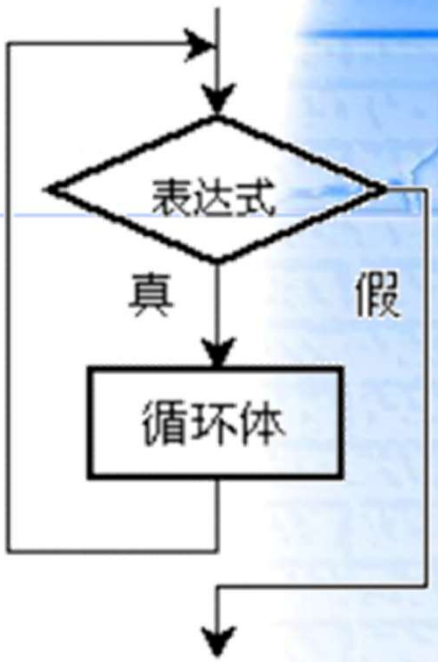
循环结构

- 循环结构是程序中一种重要的基本结构，是指：

  在一定的条件下反复执行某段程序，被反复执行的这段程序称为“循环体”

- Java中有三种语句来实现循环结构：
  - while
  - do-while
  - for

while语句

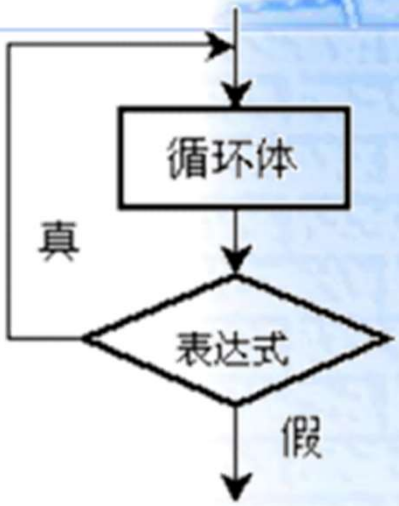


- while语句的格式如下：

```
1. while(条件表达式)
2. {
3.         循环体语句；
4. }
```

- 在循环刚开始时，会计算一次“条件表达式”的值。
  - 当条件为假时，将不执行循环体，直接跳转到循环体外，执行循环体外的后续语句；
  - 当条件为真时，便执行循环体。每执行完一次循环体，都会重新计算一次条件表达式，当条件为真时，便继续执行循环体，直到条件为假才结束循环。

do-while语句

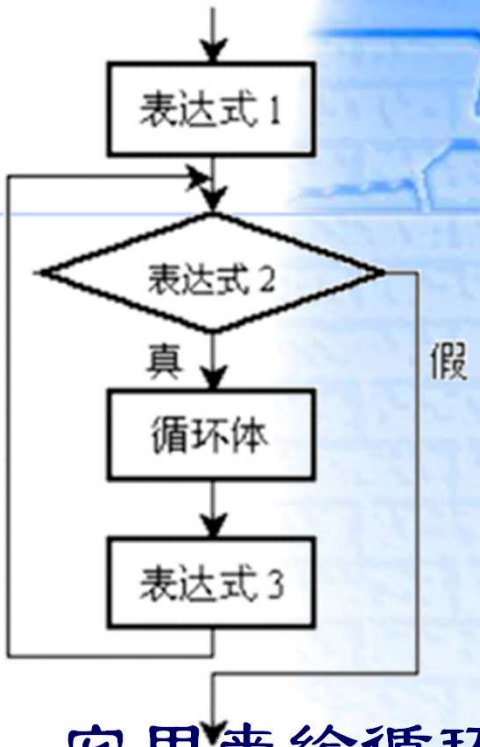


- do-while语句的格式如下：

```
1.    do
2.    {
3.          循环体语句；
4.    } while(条件表达式)
```

- do-while循环与while循环的不同在于：
  - 它先执行循环中的语句，然后再判断条件是否为真，如果为真则继续循环；如果为假，则终止循环。因此，do-while循环至少要执行一次循环语句

for语句



- for语句的格式如下：

```
1.  for (表达式1；表达式2；表达式3)
2.  {
3.      循环体语句；
4.  }
```

初始化部分（initialization）　条件部分（condition）　迭代部分（iteration）

循环部分（repetition）

- 表达式1一般是一个赋值语句，它用来给循环控制变量赋初值；
- 表达式2是一个布尔类型的表达式，它决定什么时候退出循环；
- 表达式3一般用来修改循环变量，控制变量每循环一次后按什么方式变化。
- 三个部分之间用“；”分开

for语句

```
public static void main(String[] args)
{
      int a[] = {1,2,3,4};
      for(int i:a)
      {
            System.out.println(i);
      }
}
```



```
……
      int i;
      for(i:a)
      {
            System.out.println(i);
      }
……
```



- for(声明循环变量: 数组的名字)
- 作用：定义循环变量，使其依次取数组中的每一个元素的值

循环语句之例：

计算10的阶乘~~~

```
int n=10;
long result=1;
while(n>=1)
{ result*=n--;  }
System.out.println("10!="+result);
```

while语句

```
int n=10;
long result=1;
do
{ result*=n- -; }
while(n>=1);
System.out.println("10!=“ + result);
```

do-while语句

```
long result;
int n;
for (n=10,result=1; n>0; n--)
{  result*=n;  }
System.out.println("10!= "+result);
```

for语句

循环嵌套

- 循环嵌套是指在循环体中包含有循环语句的情况.
- 三种循环语句即可以自身进行嵌套，也可以相互进行嵌套构成多重循环.
- 多重循环自内向外展开，即先执行内循环，后执行外循环。多重循环不允许相互交叉.
- 例1：打印金字塔

```
   *
  ***
 *****
*******
```

分析：把一个复杂的问题拆解成简单的问题

Example3_06

实战环节

- 实战1：

  分别输出1!、2!、3!、...10! 以及它们的和

- 实战2：

  Example3_07

  打印如下图形

```
********
*******
******
*****
****
***
**
*
```

Example3_08

跳转语句

- Java中有两种跳转语句：break语句和continue语句.
  - break用于强行退出循环，不执行循环中剩余的语句
  - continue则停止执行当前的循环，开始新的循环
- break语句和continue语句都有两种使用形式：
  - 不带标号的break语句和continue语句
  - 带标号的break语句和continue语句
  - 只使用break只能退出内循环；要想达到从内循环体中直接跳转出外循环的目的，必需与label标号语句连用

break 和 continue 之例

- 不带标号的break语句和continue语句

  依次遍历并输出1到10之间的数，若该数能被2整除，则：

  1. 使用break语句跳出循环体 【example3_09】
  2. 使用continue语句终止当前循环 【example3_10】

- 带标号的break语句和continue语句

  【example3_11】 使用带标号break语句退出循环嵌套

  【example3_12】 使用带标号continue语句继续外循环

目　录

1 基本数据类型

2 数据类型转换

3 字符集与标识符

4 运算符和表达式

5 程序流程控制

6 数　组

数 组

- 数组是程序设计使用最频繁的数据结构，它用来表示一组类型相同的有序数据。
- 这些数据使用同一个标识符命名叫数组名，数组中的每个数据叫数组元素，可通过下标来访问。
- 和变量一样，数组必须先定义，后使用。定义数组时确定数组的名称、数据类型，还要为它分配内存，初始化。
- 在Java中，数组也是一种对象。类Object中定义的方法都可以用于数组对象。

一维数组定义与赋值

- int num[]; 或 int[] num; int [] num; int []num;

  ex: float students[];

  int scores[];

- 数组在定义时不能直接分配空间

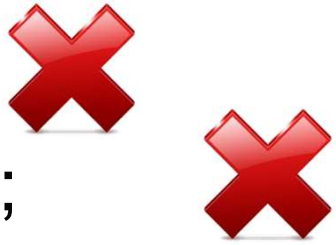
  如：int num[5];

  float students[30];

  应为：int num[];

  num = new int[5];

  或 int num[] = new int[5];



一维数组定义与赋值

- 定义时可以直接初始化

如： int num[] = {1,2,3};

int num[] = new int[]{1,2,3};

但： int num[]; num = {1,2,3};

int num[] = new int[3]{1,2,3};





int num[] = new int[ ]{1,2,3};

int num[] = new int[3];
num[0] = 1;
num[1] = 2;
num[2] = 3;

静态初始化

- 在定义数组的同时对数组元素进行初始化
- 静态初始化可用于任何元素类型，初值块中每个位置的每个元素对应一个 引用 (reference)

```
int intArray[ ]={1,2,3,4};

float [ ] heights = {84.124, 78.2, 61.5};

boolean[ ] tired = {true, false, false, true};

String[ ] names ={“Zhang", “Li", “Wang"};
```

动态初始化

- **动态初始化**是使用运算符new为数组分配空间。

数组说明的方括号中的数字表示数组元素个数:

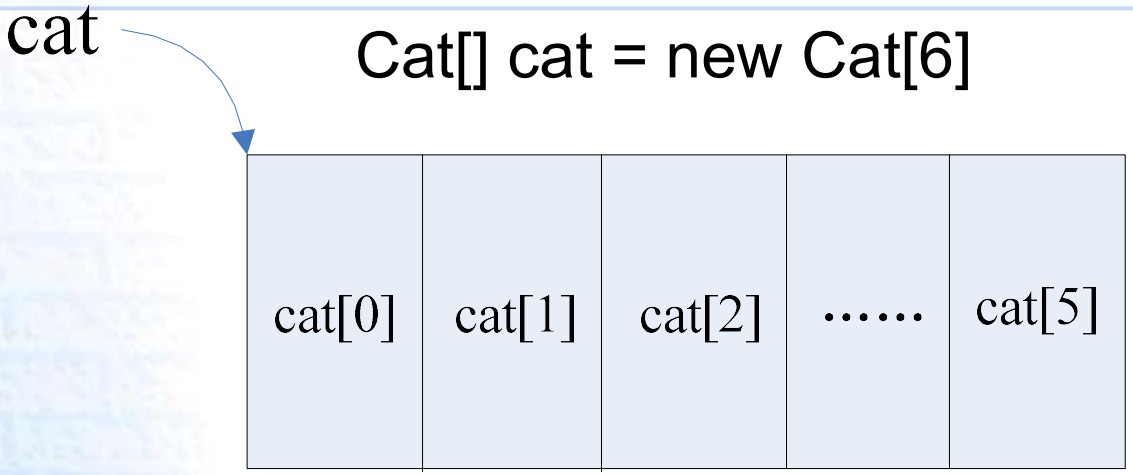
type[ ] arrayName = new type [arraySize]

type arrayName[ ] = new type [arraySize]

```
int score[ ];
score = new int[10];
```

```
int score[ ]=new int[10];
```

数组相关操作



- **数组的访问： 数组名**
  - 如： cat[0]、cat[1]

Java运行环境会自动检查数组下标是否越界

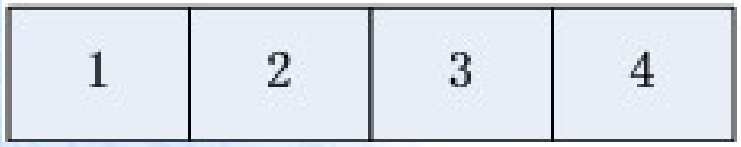
- **数组的长度： 数组名.** length
  - 如： cat.length

所有的数组都有一个属性length，这个属性存储了数组元素的个数

对象数组

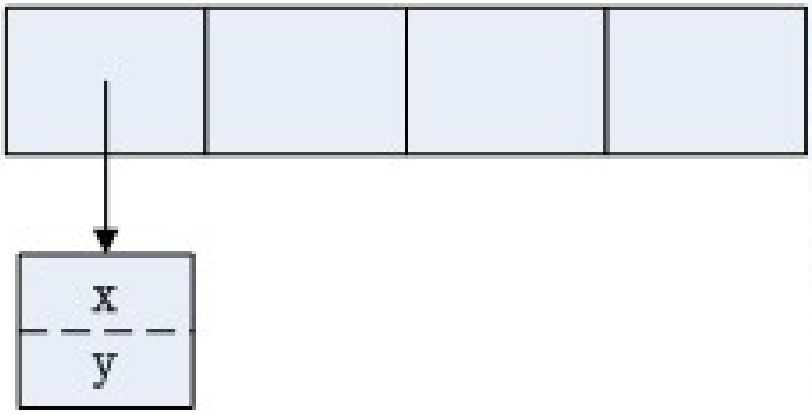
- 对象数组中每个元素的值是对某个类的对象的引用

```
int a[] = new int[]{1,2,3,4};
```




```
class Point
{
    float x;
    folat y;
}
```

```
Point points[] = new Point[4];
```



实战

- 编写“复制一维数组的程序”
  - 定义两个一维数组 a[3] 和 b[5]
  - 赋值 a[i] = i;
  - 赋值 b[j] =10* j;
  - 将a[i] 逐位赋值给b[i]
  - 将a赋值给b

Example4_01

```
int a [] = new int[3];
int b[] = new int[5];

for(int i=0; i<a.length; i++)
{   a[i] = i;  }

for(int i=0; i<b.length; i++)
{   b[i] = i;  }
```

```
for(int i=0; i<a.length; i++)
{   b[i] = a[i];  }
```

```
b=a;
```

数组的复制

- public static void **arrayCopy**(sourceArray, int index1, copyArray, int index2, int length)
- public static double[] **copyOf**(sourceArray, int newLength)，返回复制得到的新数组，新数组长度为newLength
- public staic double[] **copyOfRange**(sourceArray, int fromIndex, int toIndex)

CopyArray

二维数组定义与赋值

- int num[][]; 或 int [][] num;
  - int num[][];

    num = new int[3][4];

    int[1][2] = 50;

    int[2][3] = 70;
- 二维数组列数可以不一样
  - int [][] num = new int[3][];

    num[0] = new int[5];

    num[1] = new int[3];

    num[2] = new int[2];

    注意访问时不能越界

二维数组定义与赋值

- int num[][] = new int[][]{{1,2,3},{4,5,6}};
- int num[][] = new int[2][]{{1,2,3},{4,5,6}};
- int num[][] = new int[][]{1,2,3,4,5,6};
- int num[][] = { {1,2,3}, {4,5,6} };
- int num[][] = { {1,2,3}, {4,5}, {6} };

二维数组之例

- String s[ ][ ]=new String[2][ ];

  s[0] = new String[2];

  s[1] = new String[3];

  s[0][0] = new String(“Good”);

  s[0][1] = new String(“Luck”);

  s[1][0] = new String(“to”);

  s[1][1] = new String(“you”);

  s[1][2] = new String(“!”);

数组边界

- 二维数组行列数的检测也是通过属性length进行的，不同的是测列数时需要给定一个行标。具体办法为

  获得行数　　数组名.length

  获得列数　　数组名[行标].length

- 例如
  - int num[][] = { {1,2,3}, {4,5}, {6} };
  - num.length, num[0].length, num[1].length, num[2].length分别是多少？

实战

- 编写“复制二维数组的程序”
  - 定义两个二维数组 a[2][3] 和 b[3][4]
  - 赋值：a中每个元素的值为其所在行数和列数之和;
  - 将a赋给b;

Example4_02

```
int[][] a = new int[2][3];
int[][] b = new int[3][4];

for(int i=0; i<a.length; i++)
{
    for(int j=0; j<a[i].length; j++)
    {
        a[i][j] = i+j;
    }
}

b=a;
```

实战：枚举类型与for、switch语句

- 枚举类型的定义
  - enum WeekDay
  - {
  - sun, mon, tue, wed, thu, fri, sat
  - }
- 枚举类型值的访问
  - WeekDay day = WeekDay.mon;
  - WeekDay a[] = WeekDay.values();
- 实战：定义某水果的枚举类型，包含几种常用的水果，然后用for、switch语句访问并打印水果中部分水果的价格

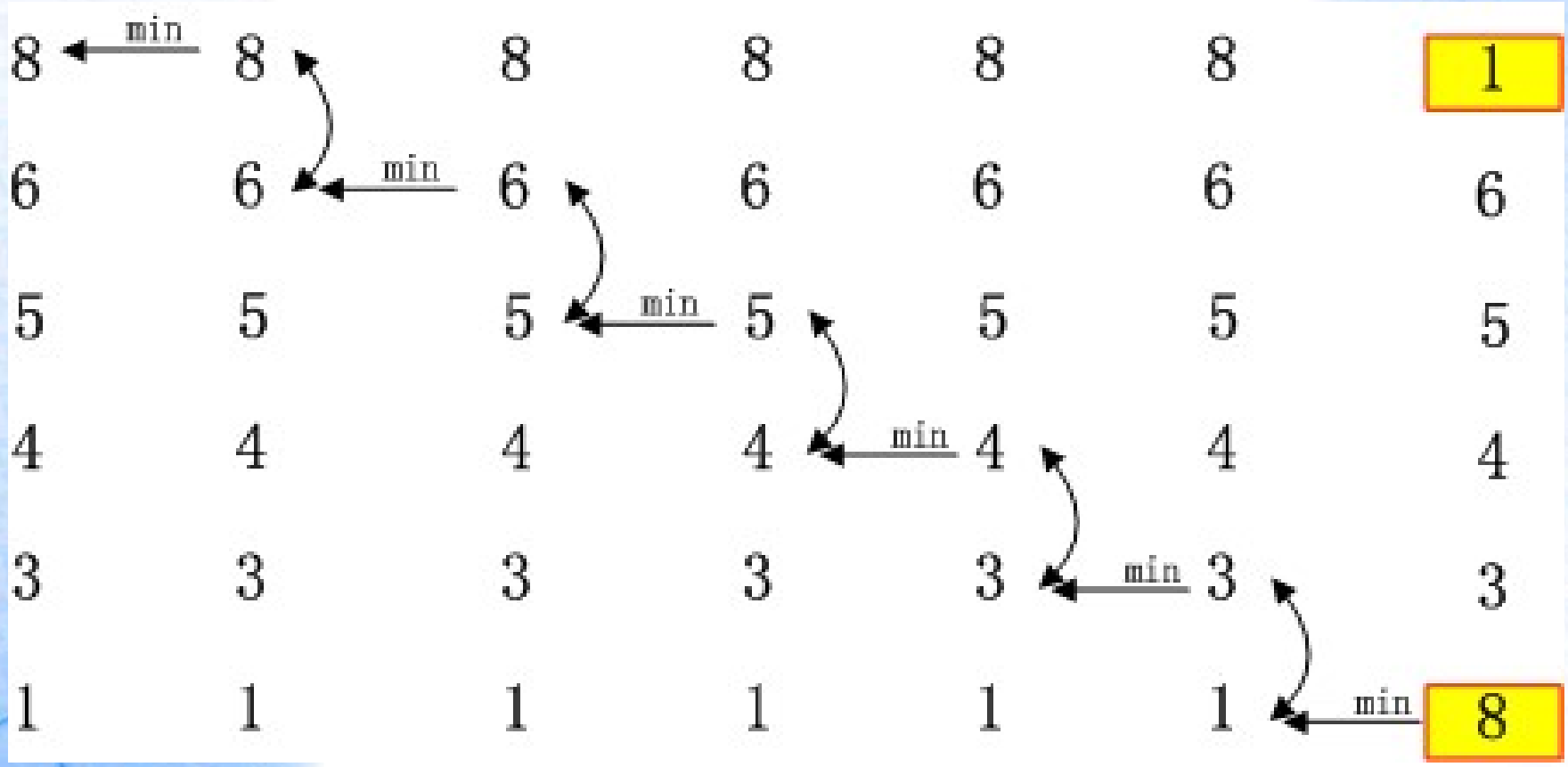
Example4_03

选择排序

- (Selection Sort)思路：
  - 从所有的数中找出最小的一个，将其放在最前面；
  - 接着在余下的数中找出最小的一个，将其放在第二位，
  - 依次类推，数列由前往后逐渐成型，直到全部记录排序完毕。

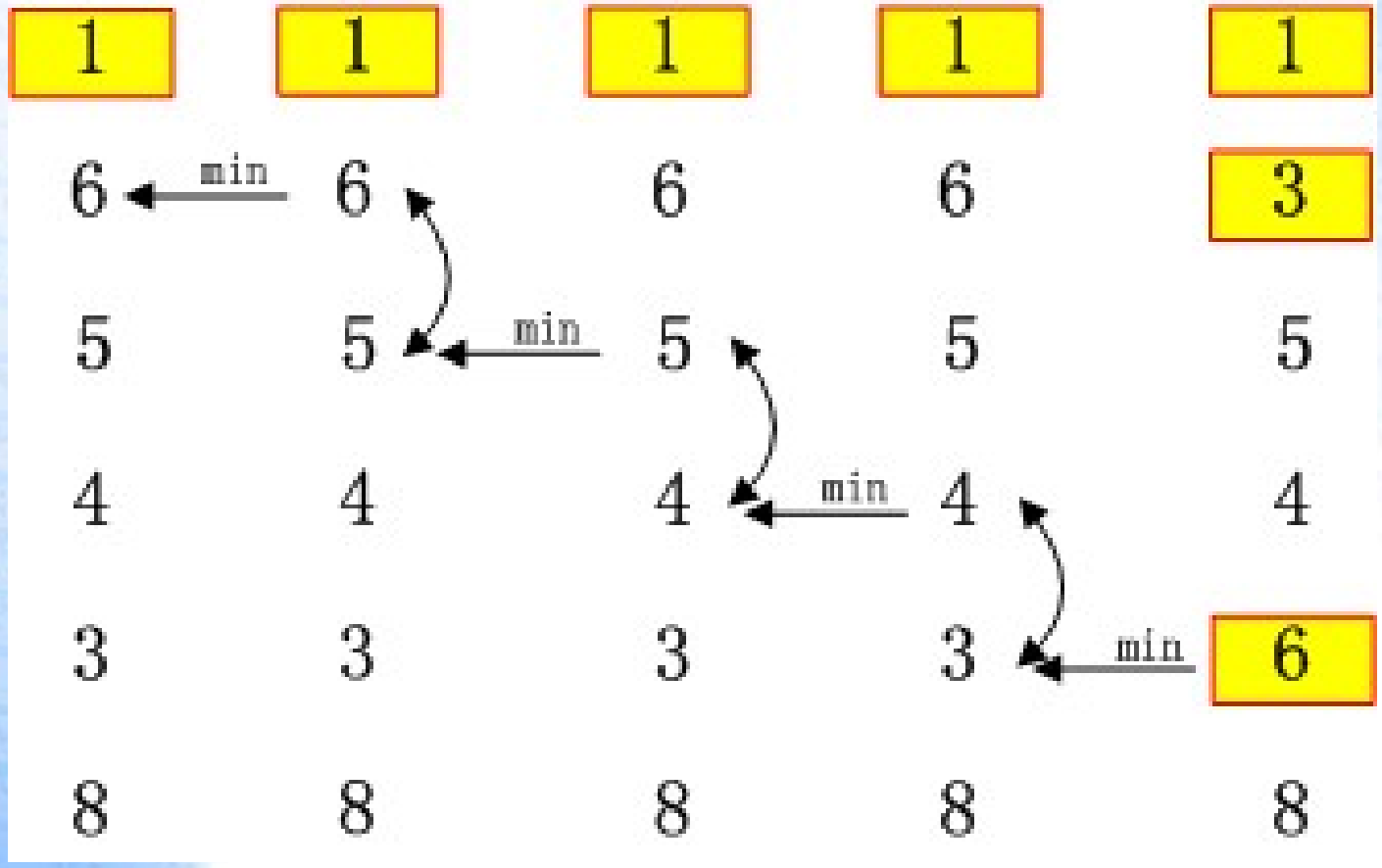
选择排序之例

- 选择法第一轮：先找出序列中最小的一个。



选择排序之例-1

- 选择法第二轮：找出余下序列中最小的一个。



冒泡排序

- 思路
  - 对相邻两个数进行比较，将较小的调到前面，两两比较一轮之后，最大的一个数被放置在最后；
  - 接着从头开始重复执行以上操作，次大的数被放置在倒数第二位，
  - 依次类推，数列由后往前逐渐成型。

冒泡例1

- 冒泡法的核心：小数上浮，大数下沉。
- 冒泡法第一轮：使最大的数放在最后一个位置上

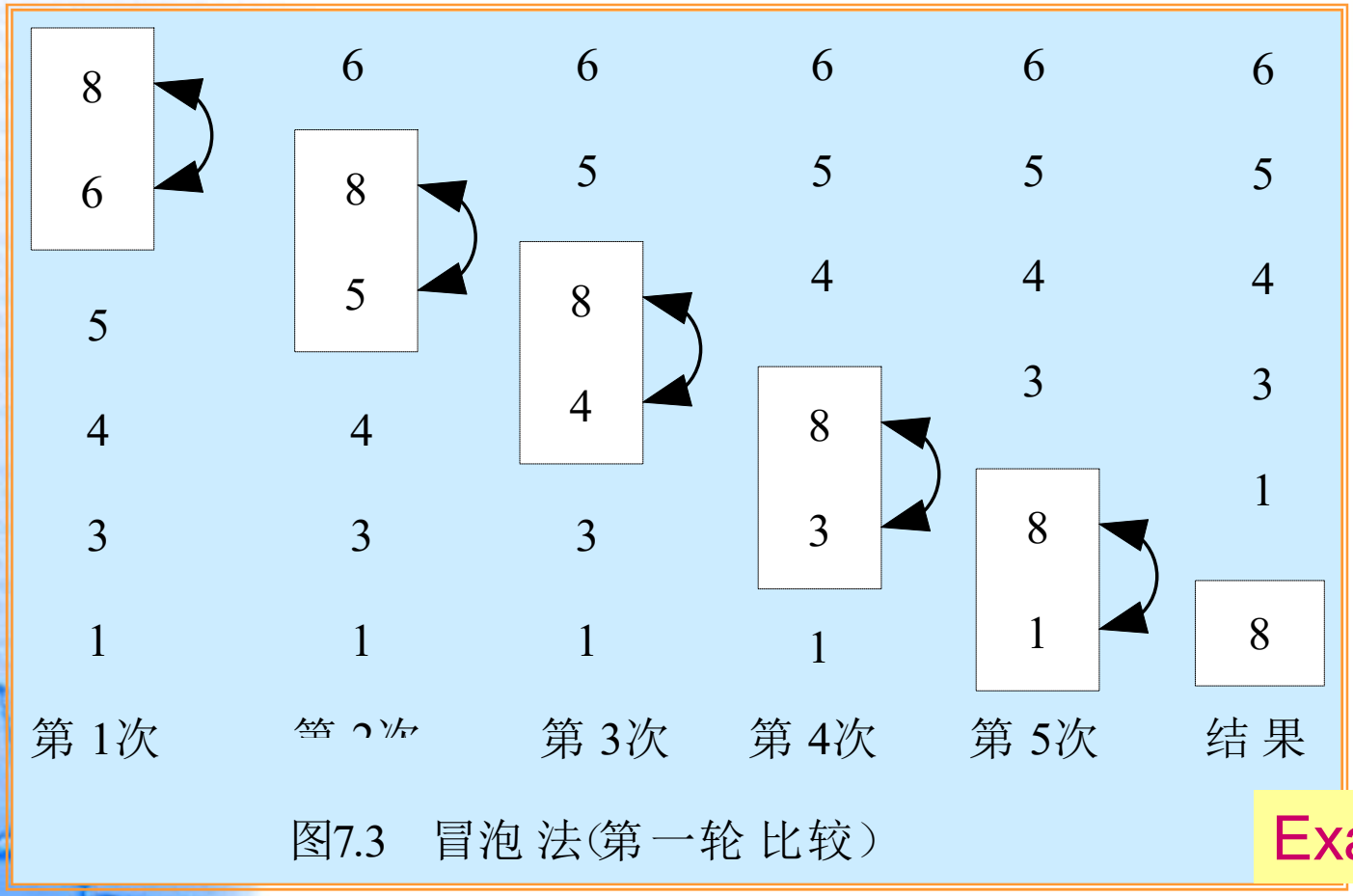


图7.3　冒泡 法(第一轮 比较)

Example4_04

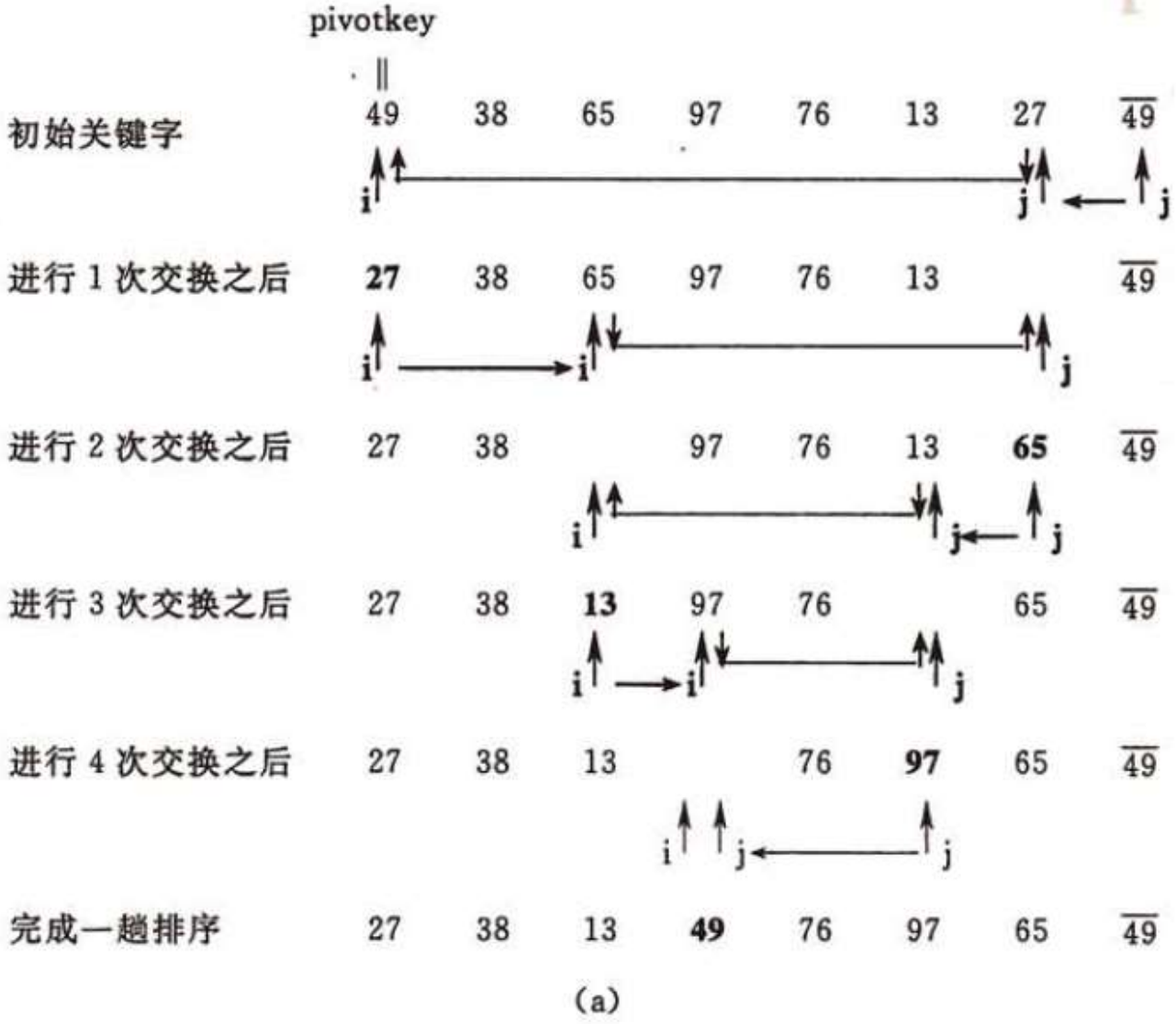
快速排序

- 对冒泡排序的一种改进
- 基本思想：

① 通过一趟排序将数据分割成独立的两部分，其中一部分的所有数据都比另外一不部分的所有数据都要小；

② 然后再按此方法对这两部分数据分别进行快速排序，整个排序过程可以递归进行，以此达到整个数据变成有序序列

快速排序


pivotkey
初始关键字 49 38 65 97 76 13 27 49
进行 1 次交换之后 27 38 65 97 76 13 49
进行 2 次交换之后 27 38 97 76 13 65 49
进行 3 次交换之后 27 38 13 97 76 65 49
进行 4 次交换之后 27 38 13 76 97 65 49
完成一趟排序 27 38 13 49 76 97 65 49
(a)

快速排序

初始状态	{49	38	65	97	76	13	27	$\overline{49}$}
一次划分之后	{27	38	13}	**49**	{76	97	65	$\overline{49}$}
分别进行快速排序	{13}	**27**	{38}					
	结束		结束		{$\overline{49}$	65}	**76**	{97}
					$\overline{\mathbf{49}}$	{65}		结束
						结束		
有序序列	{13	27	38	49	$\overline{49}$	65	76	97}

(b)

快速排序之例

- “java.util”包中，专门有一个数组类Arrays，该类提供了一些方法用于排序、查找等操作，在编制程序中可以直接使用这些方法。
- 例子：

  1. 利用Math类中的random()函数来初始化数组；
  2. 利用Arrays类中的sort()方法来对数组进行排序；
  3. 查看排序结果及排序所花的时间（Java.util包中Calendar类中的getInstance()和getTime()函数）

Example4_05

本章总结

- 本章我们学习了
  - 基本数据类型
  - 字符集与标识符
  - Unicode
  - 运算符
  - 表达式
  - 数组

Class is Dismissed!

